

HW1 Part 4 Writeup: Convert WikiHow to PDDL

1. Article Choice

I picked the wikiHow article "How to Survive in the Woods" because it describes a task that is naturally procedural and easy to imagine as a text-adventure game. The article includes concrete survival goals such as finding water, purifying water, building a fire, and making shelter. These tasks are interesting for PDDL because they have clear prerequisites and results: for example, water must be collected before it can be purified, and a fire requires dry kindling and wood before water can be boiled.

2. Portions Translated to PDDL

I translated selected parts of the article related to finding drinking water, purifying water, building a fire, building a simple shelter, and signaling for rescue. I did not try to model the entire article because wikiHow instructions contain many real-world details that would make the planning problem very large. Instead, I selected portions that could be represented as symbolic actions with clear preconditions and effects.

3. Actions, Types, and Predicates

My domain is called ``survive_in_the_woods``. The main types are ``player``, ``location``, ``direction``, ``item``, and ``water``. Some examples of predicates are ``(at ?obj ?loc)``, ``(inventory ?player ?item)``, ``(has_water_source ?loc)``, ``(untreated ?water)``, ``(treated ?water)``, ``(has_fire ?loc)``, and ``(sheltered ?player)``. Some examples of actions are ``get_water``, ``strain_water``, ``boil_water``, ``collect_kindling``, ``collect_wood``, ``build_fire_pit``, ``light_fire``, ``purify_in_sunlight``, ``cut_branches``, ``build_shelter``, and ``signal_rescue``. For example, ``boil_water`` requires the player to have water, a container, strained water, and a fire at the current location; its effect is that the water becomes treated.

4. Problems, Initial States, Goals, and Solutions

I created three PDDL problem files. In ``collect_water``, the NPC starts at camp, the canteen is at camp, and a waterfall has a water source. The goal is ``(inventory npc water)``. A solution is to get the canteen, walk to the waterfall, and use ``get_water``.

In ``purify_water``, the NPC starts with untreated water at camp. Cloth, a pot, kindling, and wood are also at camp. The goal is ``(treated water)``. A solution is to pick up the cloth, pot, kindling, and wood, strain the water, build a fire pit, light a fire, and boil the water.

In ``build_shelter``, the NPC starts at camp, a knife is at camp, and branches and leaves are in a grove. The goal is ``(sheltered npc)``. A solution is to move to the grove, collect the required materials, and build the shelter.

5. Limitations of PDDL

PDDL is useful for symbolic planning, but it is limited when converting wikiHow descriptions precisely. Many wikiHow steps involve uncertainty, judgment, quantities, time, and safety warnings. For example, "boil for 10 minutes" is a temporal and numeric condition, but the simple STRIPS-style PDDL used here does not represent time directly. The article also gives soft advice, such as looking for insects or green foliage as signs of water, but those are uncertain observations rather than guaranteed logical facts. PDDL also does not naturally model partial success, risk, or the quality of an object unless we manually add more predicates.

6. Text Adventure Game Potential

This PDDL could be used as the planning layer for a text-adventure-style survival game. The player could move between locations, collect objects, discover water sources, build a fire, purify water, and construct shelter. The planner could be used either to generate NPC behavior or to check whether a puzzle is solvable. To make it more interesting as a game, I would add hidden locations, blocked paths, limited inventory, dangerous water sources, weather conditions, and alternative solutions.

7. Using GPT or an LLM

An LLM such as GPT-4 could help convert a wikiHow article into PDDL by first extracting candidate actions, objects, preconditions, and effects from the article text. The input would be the article title, selected steps, and possibly examples of valid PDDL domains. The output could be a draft domain file, problem files, and annotations that link article phrases to PDDL elements. To fine-tune such a system, we would need paired examples of natural language instructions and human-written PDDL, including annotations showing which phrases correspond to actions, predicates, types, initial states, and goals. A human would still need to verify the result because LLMs may invent predicates, omit preconditions, or create plans that parse but do not match the real-world procedure.